ARQUITETURA TÉCNICA

Pipeline Automatizado de Análise de Tráfego Pago com Claude API

Google Ads + Meta Ads → Claude API → Relatório Diário Automatizado

Março 2026 | Documento para equipe de desenvolvimento

1. Visão Geral do Sistema

Este documento descreve a arquitetura de um pipeline automatizado que coleta dados de performance de campanhas de tráfego pago (Google Ads e Meta Ads), envia para a API do Claude (Anthropic) para análise estratégica, e entrega o relatório final via e-mail ou WhatsApp.

1.1 Objetivo

Gerar diariamente, de forma 100% automatizada, uma análise estratégica das campanhas ativas com: diagnóstico de performance, alertas de anomalias, recomendações de otimização e comparação com benchmarks internos.

1.2 Fluxo Resumido

CRON (06h) → Script Python → Google Ads API + Meta Marketing API → Consolida dados → Envia para Claude API → Recebe análise → Entrega via E-mail/Slack/WhatsApp

2. Stack Tecnológica Recomendada

Componente	Tecnologia	Motivo
Linguagem	Python 3.11+	SDKs oficiais do Google e Meta
Google Ads	google-ads-python	SDK oficial do Google
Meta Ads	facebook-business SDK	SDK oficial da Meta
IA / Análise	anthropic (Python SDK)	API do Claude, modelo claude-sonnet-4-20250514
Agendamento	CRON ou GitHub Actions	Gratuito, simples de manter
Hospedagem	VPS (DigitalOcean/AWS) ou Serverless	Baixo custo, alta confiabilidade
Entrega	SendGrid / Twilio / Slack API	E-mail, WhatsApp ou Slack

3. Coleta de Dados

3.1 Google Ads API

Pré-requisitos:

- Conta de desenvolvedor Google Ads (developer token)
- OAuth 2.0 configurado (client_id, client_secret, refresh_token)
- Customer ID da conta (ou MCC se gerencia múltiplas)

Código de referência:

```
from google.ads.googleads.client import GoogleAdsClient

client = GoogleAdsClient.load_from_storage("google-ads.yaml")
ga_service = client.get_service("GoogleAdsService")

query = """
  SELECT
    campaign.name,
    campaign.status,
    metrics.impressions,
    metrics.clicks,
    metrics.cost_micros,
    metrics.conversions,
    metrics.cost_per_conversion
  FROM campaign
  WHERE segments.date DURING YESTERDAY
  AND campaign.status = "ENABLED"
"""

response = ga_service.search(
    customer_id="SEU_CUSTOMER_ID",
    query=query
)
```

Métricas recomendadas para puxar:

Campanha: impressions, clicks, cost_micros, conversions, cost_per_conversion, ctr, average_cpc, conversion_rate, search_impression_share. Grupo de anúncio: mesmas métricas + ad_group.name. Anúncio: headline, description, final_url + métricas individuais. Palavras-chave: keyword.text, match_type, quality_score + métricas.

3.2 Meta Marketing API

Pré-requisitos:

- App criado no Meta for Developers
- Token de acesso de longa duração (System User no Business Manager)
- Ad Account ID(s) da Central de Concursos e Degrau Cultural

Código de referência:

```
from facebook_business.api import FacebookAdsApi
from facebook_business.adobjects.adaccount import AdAccount

FacebookAdsApi.init(app_id, app_secret, access_token)
account = AdAccount("act_SEU_AD_ACCOUNT_ID")

fields = [
    "campaign_name", "adset_name", "ad_name",
    "impressions", "clicks", "spend",
    "actions", "cost_per_action_type",
    "cpc", "cpm", "ctr", "frequency",
    "reach", "video_p25_watched_actions",
    "video_p50_watched_actions",
    "video_p75_watched_actions",
]

params = {
    "time_range": {"since": "2026-03-06", "until": "2026-03-06"},
    "level": "ad",
    "filtering": [{"field": "campaign.delivery_info",
                   "operator": "IN",
                   "value": ["active"]}],
}

insights = account.get_insights(fields=fields, params=params)
```

Métricas recomendadas:

Além das listadas: purchase ROAS, leads, landing_page_views, outbound_clicks, link_clicks, thumb_stop_ratio (calculado: video_p25 / impressions). Para campanhas de Central de Concursos: focar em leads (cadastros) e custo por lead. Para Degrau Cultural: focar em matrículas e custo por matrícula.

4. Integração com Claude API

4.1 Configuração

```
pip install anthropic

import anthropic
client = anthropic.Anthropic(api_key="SUA_API_KEY")
```

4.2 Estrutura do Prompt (System)

O system prompt é a peça mais crítica. Ele define o comportamento do Claude como analista de tráfego. Abaixo, um modelo sugerido:

```
SYSTEM_PROMPT = """
Você é um analista sênior de tráfego pago especializado em
marketing educacional brasileiro, especificamente em cursos
preparatórios para concursos públicos.

Você trabalha para duas marcas: Central de Concursos e Degrau Cultural.

Ao analisar os dados, siga esta estrutura:

1. RESUMO EXECUTIVO (3-5 linhas)
   - Gasto total, leads totais, CPA médio, ROAS geral

2. ALERTAS E ANOMALIAS
   - Campanhas com CPA >30% acima da média
   - Campanhas com frequência >3.0
   - Quedas de CTR >20% vs dia anterior
   - Campanhas sem conversão nas últimas 24h

3. TOP 5 CAMPANHAS (melhor CPA)
4. BOTTOM 5 CAMPANHAS (pior CPA)

5. RECOMENDAÇÕES ACIONÁVEIS
   - O que pausar, o que escalar, o que testar
   - Sugestões de copy/criativo quando pertinente

6. COMPARAÇÃO COM DIA ANTERIOR
   - Variações percentuais das métricas-chave

Regras:
- Nunca invente dados. Se faltam métricas, sinalize.
- Valores monetários em R$ (reais).
- Seja direto e acionável, sem corporativês.
- Considere o contexto do nicho de concursos públicos.
"""
```

4.3 Chamada à API

```
def analisar_campanhas(dados_consolidados: str) -> str:
    message = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=4000,
        system=SYSTEM_PROMPT,
        messages=[{
            "role": "user",
            "content": f"""
Segue o relatório de performance de ontem:

{dados_consolidados}

Analise conforme a estrutura definida.
"""
        }]
    )
    return message.content[0].text
```

4.4 Custo Estimado

Com o modelo claude-sonnet-4-20250514: input ~$3/1M tokens, output ~$15/1M tokens. Para um relatório diário com ~3.000 tokens de input (dados) e ~2.000 tokens de output (análise), o custo estimado é de aproximadamente US$ 0,04 por execução, ou cerca de US$ 1,20/mês. Extremamente baixo.

5. Formato de Consolidação dos Dados

Os dados devem ser formatados de forma legível para o Claude. Recomenda-se CSV inline ou JSON. Exemplo de formato:

```
=== GOOGLE ADS - 06/03/2026 ===
Campanha | Impressões | Cliques | CTR | CPC | Custo | Conv | CPA
[Search] INSS 2026 | 12.450 | 890 | 7.1% | R$1.82 | R$1.620 | 45 | R$36
[Search] MP-SP | 8.320 | 612 | 7.4% | R$2.10 | R$1.285 | 38 | R$33.80
...

=== META ADS - 06/03/2026 ===
Campanha | Alcance | Impress | Freq | Cliques | CTR | CPM | Gasto | Leads | CPL
[CBO] GCM SP - TOFU | 25.400 | 31.200 | 1.2 | 1.850 | 5.9% | R$18 | R$562 | 32 |
R$17.56
...
```

6. Entrega do Relatório

6.1 Via E-mail (SendGrid)

```
import sendgrid
from sendgrid.helpers.mail import Mail

def enviar_email(analise: str):
    sg = sendgrid.SendGridAPIClient(api_key="SG_API_KEY")
    message = Mail(
        from_email="relatorio@centralconcursos.com.br",
        to_emails="igor@centralconcursos.com.br",
        subject=f"[Tráfego] Análise Diária - {data_hoje}",
        html_content=converter_markdown_para_html(analise)
    )
    sg.send(message)
```

6.2 Via Slack (Webhook)

```
import requests

def enviar_slack(analise: str):
    requests.post(SLACK_WEBHOOK_URL, json={"text": analise})
```

6.3 Via WhatsApp (Twilio)

```
from twilio.rest import Client

def enviar_whatsapp(analise: str):
    client = Client(TWILIO_SID, TWILIO_TOKEN)
    client.messages.create(
        body=analise[:1600],  # limite do WhatsApp
        from_="whatsapp:+14155238886",
        to="whatsapp:+55XXXXXXXXXXX"
    )
```

7. Agendamento

7.1 Via CRON (Linux/VPS)

```
# Executar todo dia às 06:00 (horário de Brasília)
0 6 * * * cd /opt/relatorio-trafego && python main.py >> logs/$(date +\%F).log 2>&1
```

7.2 Via GitHub Actions (gratuito)

```
name: Relatorio Diario
on:
  schedule:
    - cron: "0 9 * * *"  # 09:00 UTC = 06:00 BRT
jobs:
  run:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -r requirements.txt
      - run: python main.py
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
          META_ACCESS_TOKEN: ${{ secrets.META_ACCESS_TOKEN }}
          # ... demais segredos
```

8. Estrutura de Arquivos do Projeto

```
relatorio-trafego/
├── main.py                  # Orquestrador principal
├── collectors/
│   ├── google_ads.py        # Coleta Google Ads API
│   └── meta_ads.py          # Coleta Meta Marketing API
├── analysis/
│   ├── claude_client.py     # Integração com Claude API
│   └── prompts.py           # System prompts e templates
├── delivery/
│   ├── email_sender.py      # SendGrid
│   ├── slack_sender.py      # Slack webhook
│   └── whatsapp_sender.py   # Twilio
├── config/
│   ├── settings.py          # Configurações gerais
│   └── google-ads.yaml      # Credenciais Google Ads
├── data/
│   └── historico/            # Relatórios anteriores (para comparação)
├── logs/
├── requirements.txt
├── .env                         # Variáveis de ambiente (NUNCA commitar)
└── .github/workflows/daily.yml  # GitHub Actions (se usar)
```

9. Dependências (requirements.txt)

```
anthropic>=0.40.0
google-ads>=25.0.0
facebook-business>=20.0.0
python-dotenv>=1.0.0
sendgrid>=6.11.0        # se usar e-mail
twilio>=9.0.0           # se usar WhatsApp
requests>=2.31.0        # se usar Slack
pandas>=2.2.0           # para consolidação de dados
```

10. Credenciais Necessárias

Serviço	O que precisa	Onde conseguir
Claude API	API Key	console.anthropic.com
Google Ads	Developer Token + OAuth2 credentials + Refresh Token	ads.google.com/aw/apicenter + console.cloud.google.com
Meta Ads	System User Token + Ad Account ID	business.facebook.com > Config > System Users
SendGrid	API Key	app.sendgrid.com
Twilio (opc)	Account SID + Auth Token	console.twilio.com

11. Coleta Automatizada de Criativos

Esta seção detalha como puxar automaticamente os criativos (imagens e thumbnails de vídeo) das campanhas ativas e enviá-los para o Claude junto com os KPIs, permitindo análise integrada de performance + qualidade visual/copy.

11.1 Meta Ads — Criativos via Marketing API

A Meta Marketing API oferece acesso completo aos assets criativos. O campo ad_creative retorna URLs de imagens, thumbnails de vídeo e todos os textos do anúncio.

Campos disponíveis:

Campo	Descrição	Uso na análise
image_url	URL da imagem do anúncio	Enviar para Claude via vision
thumbnail_url	Thumbnail do vídeo	Avaliar frame inicial / hook visual
video_id	ID do vídeo (para puxar thumbnails)	Endpoint: /{video_id}?fields=thumbnails
body	Texto principal do anúncio	Análise de copy
title	Título / headline	Análise de gancho
object_story_spec	Estrutura completa do post	CTA, link, formato

Código de referência — Coleta Meta:

```
from facebook_business.adobjects.ad import Ad
from facebook_business.adobjects.adcreative import AdCreative
import requests, base64

def coletar_criativos_meta(account):
    """Coleta criativos de todos os ads ativos."""
    ads = account.get_ads(
        fields=["name", "creative{image_url,thumbnail_url,
                 video_id,body,title,object_story_spec}",
                 "insights{impressions,clicks,spend,
                 actions,cost_per_action_type,ctr}"],
        params={"effective_status": ["ACTIVE"]},
    )

    criativos = []
    for ad in ads:
        creative = ad.get("creative", {})
        image_url = creative.get("image_url") \
                    or creative.get("thumbnail_url")

        # Baixar imagem e converter para base64
        img_b64 = None
```

```
        if image_url:
            resp = requests.get(image_url)
            if resp.status_code == 200:
                img_b64 = base64.standard_b64encode(
                    resp.content).decode("utf-8")

        # Para vídeos, puxar thumbnail específica
        video_id = creative.get("video_id")
        if video_id and not img_b64:
            vid = AdVideo(video_id).api_get(
                fields=["thumbnails"]
            )
            thumb_url = vid["thumbnails"]["data"][0]["uri"]
            resp = requests.get(thumb_url)
            if resp.status_code == 200:
                img_b64 = base64.standard_b64encode(
                    resp.content).decode("utf-8")

        criativos.append({
            "ad_name": ad["name"],
            "copy_body": creative.get("body", ""),
            "copy_title": creative.get("title", ""),
            "image_base64": img_b64,
            "kpis": ad.get("insights", {}).get("data", [{}])[0],
        })

    return criativos
```

11.2 Google Ads — Criativos via API

No Google Ads, o acesso a criativos visuais depende do tipo de campanha. Campanhas de Search retornam apenas assets de texto (headlines e descriptions). Campanhas de Performance Max, Display e Demand Gen retornam imagens e vídeos associados aos asset groups.

Tipo de Campanha	Criativos Disponíveis	Como acessar
Search	Headlines, descriptions, sitelinks (somente texto)	GAQL: ad_group_ad.ad.responsive_search_ad
Performance Max	Imagens, vídeos, headlines, descriptions, logos	GAQL: asset_group_asset + asset.image_asset
Display	Imagens, banners, responsive display ads	GAQL: ad_group_ad.ad.responsive_display_ad
Demand Gen	Imagens, vídeos, carrosséis	GAQL: asset_group_asset

Código de referência — Coleta Google Ads:

```
def coletar_criativos_google(client, customer_id):
    """Coleta assets visuais de PMax e Display."""
    ga_service = client.get_service("GoogleAdsService")

    # Assets de Performance Max
    query_pmax = """
      SELECT
        campaign.name,
        asset_group.name,
        asset.image_asset.full_size.url,
        asset.image_asset.file_size,
        asset.name,
        asset.type,
        asset_group_asset.performance_label
      FROM asset_group_asset
      WHERE campaign.status = "ENABLED"
        AND asset.type IN ("IMAGE", "VIDEO")
    """

    # Responsive Search Ads (somente texto)
    query_rsa = """
      SELECT
        campaign.name,
        ad_group.name,
        ad_group_ad.ad.responsive_search_ad.headlines,
        ad_group_ad.ad.responsive_search_ad.descriptions,
        ad_group_ad.ad.final_urls,
        metrics.impressions,
        metrics.clicks,
        metrics.conversions,
        metrics.cost_micros
```

```
      FROM ad_group_ad
      WHERE campaign.status = "ENABLED"
        AND ad_group_ad.status = "ENABLED"
        AND segments.date DURING YESTERDAY
    """

    # Executar queries e baixar imagens
    response = ga_service.search(
        customer_id=customer_id, query=query_pmax
    )
    for row in response:
        img_url = row.asset.image_asset.full_size.url
        perf_label = row.asset_group_asset.performance_label
        # Baixar e converter para base64 (mesmo fluxo do Meta)
```

11.3 Enviando Criativos para o Claude (Vision)

A API do Claude suporta análise de imagens (vision). O criativo é enviado como base64 junto com os KPIs no mesmo request, permitindo análise integrada.

Código de referência:

```
def analisar_criativo(img_base64, copy_text, kpis):
    """Envia criativo + KPIs para análise integrada."""
    message = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=2000,
        system="""Você é um analista de criativos para campanhas
de tráfego pago no nicho de concursos públicos.
Avalie o criativo considerando:
1. HOOK VISUAL - A imagem prende atenção no feed?
2. HIERARQUIA - A informação principal está clara?
3. CTA - O chamado para ação está visível e direto?
4. COPY - O texto está alinhado com o visual?
5. PERFORMANCE - O que os KPIs indicam sobre o criativo?
6. RECOMENDAÇÕES - O que mudar para melhorar?
Seja direto e acionável.""",
        messages=[{
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "source": {
                        "type": "base64",
                        "media_type": "image/jpeg",
                        "data": img_base64
                    }
                },
                {
                    "type": "text",
                    "text": f"""
Copy do anúncio: {copy_text}

KPIs de performance:
{kpis}

Analise este criativo conforme a estrutura definida.
"""
                }
            ]
        }]
    )
    return message.content[0].text
```

11.4 Fluxo Completo Integrado

O pipeline final funciona assim: o script coleta os KPIs + criativos de todas as campanhas ativas, gera primeiro o relatório geral de performance (seção 4), depois faz uma segunda passada analisando os criativos dos Top 5 e Bottom 5 anúncios por CPA. Isso evita custo desnecessário com análise de criativos de campanhas irrelevantes.

```
# Fluxo simplificado do main.py atualizado

# 1. Coleta KPIs
dados_google = coletar_kpis_google()
dados_meta = coletar_kpis_meta()

# 2. Análise geral de performance
relatorio = analisar_campanhas(consolidar(dados_google, dados_meta))

# 3. Identificar Top 5 e Bottom 5 por CPA
top5, bottom5 = rankear_por_cpa(dados_google + dados_meta)

# 4. Coletar criativos apenas desses 10 anúncios
criativos = coletar_criativos(top5 + bottom5)

# 5. Analisar cada criativo
analises_criativos = []
for c in criativos:
    if c["image_base64"]:
        analise = analisar_criativo(
            c["image_base64"], c["copy"], c["kpis"]
        )
        analises_criativos.append(analise)

# 6. Montar relatório final e enviar
relatorio_final = relatorio + "\n\n" + "\n".join(analises_criativos)
enviar_email(relatorio_final)
```

11.5 Custo Adicional Estimado

Cada imagem enviada via vision consome aproximadamente 1.600 tokens (para imagens até 768px). Analisando 10 criativos por dia (Top 5 + Bottom 5), o custo adicional é de cerca de US$ 0,05/dia ou US$ 1,50/mês. Somado ao relatório de KPIs (US$ 1,20/mês), o custo total do pipeline fica em torno de US$ 2,70/mês.

12. Evoluções Futuras

1. **Histórico comparativo:** Salvar cada relatório em JSON e incluir dados da semana anterior no prompt para o Claude fazer análise de tendência.
2. **Dashboard web:** Interface simples com Flask/Streamlit para visualizar relatórios e interagir com o Claude em tempo real sobre os dados.
3. **Integração TikTok Ads:** Adicionar collector para TikTok Marketing API no mesmo pipeline.
4. **Alertas em tempo real:** Execução a cada 4h com alertas apenas quando houver anomalias, sem gerar relatório completo.